



Arab Academy for Science and Technology

College of Computing & Information Technology

Made by: Yehia Ahmed Yehia 221006879

Submitted to: Dr. Hisham Dahshan

Advanced Cryptography Project

Walkthrough:

Module 1 : 1) OTP & Perfect Secrecy

--- OTP ---

OTP Demo

Plaintext : b'Hello, Advanced Cryptography!'

Key (hex) : 422957e1a2009aaff08b87ecc4080c2d4b65d247b0d846893480c5be8c

Ciphertext: 0a4c3b8dcd2cbaee94fde682a76d680d0817ab37c4b721fb55f0adc7ad

Decrypted : b'Hello, Advanced Cryptography!'

Round-trip OK

CT1: 0a4c3b8dcd2cbaee94fde682a76d680d0817ab37c4b721fb55f0adc7ad

CT2: 0a4c3b8dcd2cbaee94fde682a76d680d0817ab37c4b721fb55f0adc7ad

Same key produces same ciphertext

Different key produces different ciphertext

Perfect Secrecy (Shannon's theorem):

For any ciphertext c and any message m' , there exists a key k' such that $\text{Dec}(c, k') = m'$.

Real message : b'Attack at dawn!!'

Ciphertext (hex): 7ded720ba39a1698f0392d5fe9218178

Fake message : b'Nothing to see.\x00'

Fake key (hex) : 33820663caf471b884560d2c8c44af78

Decrypted w/fake: b'Nothing to see.\x00'

Same ciphertext maps to a different message with a different key.

All OTP tests passed.

Module 2 : 2) AES-CBC (CPA-Safe)

```
--- AES-CBC ---

AES-CBC Demo

AES Key (hex): 8406c7ec91e78855d0c80615067c7042

Plaintext : b'Hello, AES-CBC with PKCS#7 padding!'
Ciphertext: 346801b8bbc198c05f5eaa54ffa7c89991292fcbee5132ce352dfc4639b3b06e071897fafd34608c5495625df8f7f6dac12579e84577e22b9fb5585b4d443601
CT length : 64 bytes (16 IV + 48 encrypted)
Decrypted : b'Hello, AES-CBC with PKCS#7 padding!'
Round-trip OK

CT1: 10d3b6ce5b228adb8d7e2d36432b8bd264acd02aba466134a1a323b0d2a6d12cab20ec4f8342c00b43031b89d3cd6da41d59c9312b801ebe86ebab3ald725fea
CT2: e01a3a2ef94086af7db69c71326df1f378499e0fda9d5f5d14cf4ada942e0368554ff1cc47d2d5f3891e9ef5f4705af76741c9f4e88dd672731d06811b2b462d
IV1: 10d3b6ce5b228adb8d7e2d36432b8bd2
IV2: e01a3a2ef94086af7db69c71326df1f3
Same plaintext gives different ciphertexts (random IV = CPA security)

PKCS#7 padding:
 1 bytes -> 16 bytes, pad=0x0f (15 bytes)
15 bytes -> 16 bytes, pad=0x01 (1 bytes)
16 bytes -> 32 bytes, pad=0x10 (16 bytes)
31 bytes -> 32 bytes, pad=0x01 (1 bytes)

Valid ciphertext -> oracle = True
Tampered ciphertext -> oracle = False

All AES-CBC tests passed.
```

Module 3 : 3) Hash Functions

```

--- Hash Functions ---

Hash Functions Demo

SHA-256("Hello, World!") = dffd6021bb2bd5b0af676290809ec3a5...
SHA-256("Hello, World!") = dffd6021bb2bd5b0af676290809ec3a5...
SHA-256("Hello, World") = 03675ac53ff9cd1535ccc7dfcdfa2c45...
SHA-256("") = e3b0c44298fc1c149afbf4c8996fb924...
SHA-256 is deterministic
Avalanche: 1-byte change flips 131/256 bits (51.2%)

Merkle-Damgard Construction:

  IV -> compress(H0, M1) -> compress(H1, M2) -> ... -> Hash

Message: "Merkle-Damgard construction demo for Advanced Cryptography course!"
Length: 66 bytes

Step 1 - Padding: 128 bytes (2 blocks)
Step 2 - IV: 6a09e667bb67ae853c6ef372a54ff53a...
Step 3 - Processing 2 block(s):
  Block 1: H_0 -> compress -> H_1 = 1d51b5a472b9c004...
  Block 2: H_1 -> compress -> H_2 = 8235fa3c7a9f486a...
Step 4 - Simulated hash: 8235fa3c7a9f486adf0e2456b7bdbc6c18898010716b00c0511a72335f60e629
Real SHA-256 hash:      e9df8c31e141c927398371d90bfc63987732a25159044738362e0c3733756fc0

Note: simulated hash differs because we used SHA256(state||block)
as compression, not the actual SHA-256 compression function.
The point is to show the structure.

Commitment Scheme:
Message: "I predict the coin will land on heads."
Nonce:   84dad8514f04fe5b20c10d098792ef13...
Commit:  32f7184247d61c79d640b6fd1392ca7a201564e3953f4564a44e54e314dc322b
Verify correct: True, wrong msg: False, wrong nonce: False

SHA-256 Benchmarks:
  Size      Avg Time      Throughput
  16 B      0.0045 ms      3.36 MB/s
  64 B      0.0049 ms      12.48 MB/s
  256 B     0.0053 ms      46.04 MB/s
  1 KB      0.0080 ms     121.66 MB/s
  4 KB      0.0176 ms     222.53 MB/s
  16 KB     0.0558 ms     280.14 MB/s
  64 KB     0.2096 ms     298.20 MB/s
  256 KB    0.8114 ms     308.12 MB/s
  1 MB      3.2307 ms     309.53 MB/s

All hash tests passed.

```

Module 3 : 4) RSA Basics

--- RSA ---

RSA Demo

Generating RSA keys (512-bit primes)...

p (512 bits): 9389217989509459645720299973636558792802...

q (512 bits): 1147687330227700806150391303144957833828...

n (1024 bits): 1077588652730601225587253723845140797390...

e: 65537

d (1023 bits): 7506141505684525771592187580642159778703...

e * d = 1 (mod phi(n)) verified

Plaintext: "RSA works!"

m = 388769824235561065083681

c = 5302347888596876287835629101337245378344...

Decrypted: "RSA works!"

Round-trip OK

Public key saved to public_key.json

Import verified

Generating vulnerable RSA keys (e=3)...

n (1024 bits): 1032020894590422588485327077887567711711...

e: 3

Small message: "Hi" (m = 18537)

$m^3 = 6369690780153$

$m^3 < n$? True

Since $m^3 < n$, modular reduction does nothing.

Attacker can just take the cube root of c.

Cube root of c = 18537 -> "Hi"

Message recovered without private key!

Public key saved to vulnerable_public_key.json

Normal decryption also works

All RSA tests passed.

Summary:

OTP: OK

AES-CBC: OK

Hash Functions: OK

RSA: OK

All modules passed.